

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

04 - HTML5

Prof. Alex Kutzke

Agosto de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Escrever um documento HTML5 válido a partir do zero;
2. Escolher o elemento certo pelo significado do conteúdo, e não pela aparência;
3. Explicar como o navegador transforma o arquivo na árvore que exibe;
4. Construir formulários e relacionar cada campo com a requisição HTTP;
5. Aplicar validação nativa e dizer por que ela não substitui o servidor;
6. Corrigir decisões de marcação que quebram a acessibilidade;
7. Validar o documento no W3C e interpretar os erros.

Roteiro

Fundamentos

O documento

Conteúdo

Semântica e acessibilidade

Formulários

Prática

Exercícios

O que o navegador recebe

Na aula passada, o corpo da resposta veio como texto:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

```
<!DOCTYPE html>
<html lang="pt-br">
...
```

O servidor manda **texto**. O navegador interpreta esse texto, monta uma estrutura na memória e desenha a tela a partir dela.

HyperText Markup Language

Linguagem de marcação

O conteúdo continua sendo texto comum. As **marcas** dizem o que cada trecho é: isto é um título, aquilo é um link, aquele outro é um parágrafo.

HTML descreve conteúdo e estrutura. Não há variável, condicional nem laço: a linguagem anota um texto, e só.

O que o Content-Type: `text/html` faz é avisar o navegador de que aquele corpo deve ser interpretado, e não exibido literalmente.

Elemento, tag e conteúdo

```
<p>Este é um parágrafo.</p>
```

- `<p>`: tag de abertura;
- `</p>`: tag de fechamento;
- entre as duas: o conteúdo do elemento.

Elementos **vazios** não têm conteúdo nem fechamento:

```
  
<br>  
<hr>
```

Atributos

```
<a href="https://www.ufpr.br" title="Página da UFPR">UFPR</a>
```

- ficam na tag de **abertura**;
- nome fixo, definido pela especificação;
- valor entre **aspas duplas**.

Atributo inventado não gera erro. Simplesmente nada acontece.

Aninhamento

```
<p>Um <strong>aviso <em>importante</em></strong> no texto.</p>
```

O que abre por último fecha primeiro.

Marcação cruzada

`<strong><em></strong></em>` está errado. O navegador **não** recusa: ele adivinha e segue em frente.

O navegador nunca reclama

Diferente de um compilador, o navegador jamais recusa uma página. Diante de marcação quebrada, aplica regras de recuperação de erro e exibe algo.

O resultado típico:

- funciona na sua máquina;
- quebra na do colega;
- fica ilegível para um leitor de tela.

Por isso existe o validador do W3C, no fim da aula.

Estrutura mínima

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,
    ↪ initial-scale=1">
  <title>Catálogo de Produtos</title>
</head>
<body>
  <h1>Catálogo de Produtos</h1>
</body>
</html>
```

Cada linha do esqueleto tem função

- `<!DOCTYPE html>`: sem ele, o navegador entra em modo de compatibilidade com os anos 1990 e o layout muda;
- `lang="pt-br"`: pronúncia do leitor de tela, classificação em buscadores;
- `charset="utf-8"`: sem isso, os acentos viram símbolos estranhos;
- `viewport`: sem isso, o celular finge ser um monitor e o site aparece reduzido;
- `<title>`: aba, favorito e resultado de busca.

head: informação sobre o documento. **body**: o que aparece.

Do arquivo para a árvore

```
<body>
  <h1>Produtos</h1>
  <ul>
    <li>Teclado</li>
    <li>Monitor</li>
  </ul>
</body>
```

```
body
|-- h1
|   |-- "Produtos"
|-- ul
|     |-- li
|       |-- "Teclado"
|       |-- li
|         |-- "Monitor"
```

Essa árvore é o **DOM** (*Document Object Model*).

Por que a árvore importa desde já

- o CSS estiliza a árvore, e o JavaScript manipula a árvore. Nenhum dos dois enxerga o texto do seu arquivo;
- aninhar errado desloca o galho inteiro;
- as ferramentas do navegador mostram a árvore **já corrigida**, e por isso ela às vezes difere do que você escreveu.

O DOM volta em detalhe na aula de JavaScript.

Títulos

```
<h1>Catálogo de Produtos</h1>  
<h2>Periféricos</h2>  
<h3>Teclados</h3>
```

De h1 a h6, em ordem de importância.

Não pule níveis

Escolher h3 porque a letra sai menor quebra o índice da página. O tamanho se resolve no CSS, semana que vem.

Parágrafos, quebras e ênfase

```
<p>Preço <strong>promocional</strong>, válido <em>hoje</em>.</p>
```

- o HTML ignora quebras de linha e espaços repetidos do arquivo. Quem separa parágrafos é a marcação;
- `<br>` serve para quebra que faz parte do conteúdo, como endereço. Nunca para criar espaço vertical;
- `<strong>` e `<em>` têm significado; `<b>` e `<i>` só mudam a aparência.

Listas

```
<ul>
```

```
  <li>Teclado</li>
```

```
  <li>Monitor</li>
```

```
</ul>
```

```
<ol>
```

```
  <li>Abrir o editor</li>
```

```
  <li>Criar o arquivo</li>
```

```
</ol>
```

ul quando a ordem não importa, ol quando importa. Filho direto de lista é sempre li.

Menu de navegação é uma lista de links, e deve ser marcado como tal.

Links e caminhos

```
<a href="catalogo.html">Ver o catálogo</a>
<a href="../index.html">Voltar</a>
<a href="#contato">Ir para o contato</a>
<a href="https://www.ufpr.br" target="_blank"
↪  rel="noopener">UFPR</a>
```

- relativo entre páginas do seu próprio site;
- caminho do seu disco (file:///C:/Users/...) funciona só na sua máquina;
- o texto do link precisa dizer para onde ele leva. Dez links escritos *clique aqui* não informam nada a quem usa leitor de tela.

Imagens

```

```

O `alt` é lido em voz alta, aparece quando a imagem falha e é indexado por buscadores.

Decoração

`alt=` vazio faz o leitor de tela ignorar a imagem. **Omitir** o atributo faz ele ler o nome do arquivo.

Figura com legenda

```
<figure>
  
  <figcaption>Teclado ABNT2, switches marrons.</figcaption>
</figure>
```

alt descreve a imagem para quem não a vê. figcaption é a legenda, lida por todos.

Tabelas

```
<table>
  <caption>Especificações</caption>
  <thead>
    <tr><th scope="col">Componente</th><th
      ↳ scope="col">Preço</th></tr>
  </thead>
  <tbody>
    <tr><td>Processador</td><td>R$ 890,00</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td>R$ 1.210,00</td></tr>
  </tfoot>
</table>
```

scope permite ao leitor de tela anunciar *Processador, preço, 890 reais*.

Tabela é para dado tabular

Antes do CSS, o mercado inteiro usava tabela para posicionar coisas na tela. Hoje isso é erro de acessibilidade: o leitor de tela anuncia linhas e colunas que não existem.

Layout se faz com CSS, na aula que vem.

Tudo poderia ser div

`<div>` é um elemento genérico, sem significado nenhum. Uma página feita só de `div` funciona.

E vira uma pilha de caixas indistinguíveis para quem não enxerga o layout: o leitor de tela perde os pontos de referência, e o buscador não sabe o que é conteúdo e o que é menu.

Elementos estruturais

- `<header>`: cabeçalho da página ou de uma seção;
- `<nav>`: bloco de links de navegação;
- `<main>`: conteúdo principal, único por página;
- `<article>`: conteúdo que faz sentido sozinho;
- `<section>`: agrupamento temático, com título próprio;
- `<aside>`: conteúdo lateral, relacionado mas não essencial;
- `<footer>`: rodapé da página ou da seção.

Uma página típica

```
<body>
  <header>
    <h1>Catálogo</h1>
    <nav><ul><li><a href="index.html">Início</a></li></ul></nav>
  </header>
  <main>
    <section>
      <h2>Destaques</h2>
      <article><h3>Teclado mecânico</h3><p>ABNT2.</p></article>
    </section>
  </main>
  <footer><p>Fulano de Tal, GRR20260000.</p></footer>
</body>
```

Como escolher o elemento

Pergunte o que o bloco **é**, e não onde ele aparece na tela.

- recorto e publico sozinho? `article`;
- agrupa assuntos sob um título? `section`;
- nenhum elemento descreve o papel e eu só preciso de um agrupamento para estilizar? aí sim `div`, ou `span` dentro de uma linha de texto.

Acessibilidade: o que já resolve a maior parte

- lang no elemento html;
- títulos em ordem, sem pular níveis;
- alt em toda imagem;
- texto de link que diz para onde leva;
- <label> ligado a cada campo de formulário;
- elementos estruturais no lugar de div genérico.

Quase tudo sai de marcação correta, sem trabalho extra. Sites públicos no Brasil são obrigados a isso pelo eMAG e pela Lei Brasileira de Inclusão.

Teclado e ARIA

Teclado

Link, botão e campo são alcançáveis por Tab porque são elementos nativos. Uma `div` com clique preso por JavaScript não é, e quem usa só teclado não consegue ativá-la.

ARIA

Atributos como `aria-label` e `role` complementam o significado quando o HTML não dá conta. A primeira regra da especificação: não use ARIA quando existe elemento HTML com aquele papel.

Teste barato: percorra a sua página só com Tab.

Formulário mínimo

```
<form action="processa.php" method="post">
  <label for="nome">Nome completo</label>
  <input type="text" id="nome" name="nome" required>

  <label for="email">E-mail</label>
  <input type="email" id="email" name="email" required>

  <button type="submit">Enviar</button>
</form>
```

Aqui a aula de HTTP encontra a de HTML.

Os quatro atributos que importam

- `action`: a URL que recebe os dados. Vazio, é a própria página;
- `method`: com `get`, os dados vão na URL, à vista e no histórico; com `post`, vão no corpo da requisição;
- `name`: o nome do dado no envio. **Campo sem name não é enviado;**
- `id`: identifica o elemento no documento e liga o `<label>` ao campo pelo `for`.

O que chega ao servidor é o par `name=valor`.

Por que o label associado

- o leitor de tela anuncia o rótulo ao chegar no campo;
- o clique no texto move o foco para o campo, o que ajuda quem tem dificuldade motora.

Rótulo solto num `<p>` ao lado não produz nenhum dos dois efeitos.

placeholder é exemplo dentro do campo, e some ao digitar. Não substitui o `<label>`.

Tipos de campo

<code><input type="text"></code>	<code><input type="date"></code>
<code><input type="email"></code>	<code><input type="tel"></code>
<code><input type="password"></code>	<code><input type="url"></code>
<code><input type="number"></code>	<code><input type="file"></code>

O type muda o teclado no celular, o controle exibido e a validação aplicada.

password

Esconde os caracteres na tela e nada mais. O valor viaja como os demais. Quem protege o envio é o HTTPS.

Escolha entre opções

```
<fieldset>
  <legend>Categoria</legend>
  <input type="radio" id="p" name="categoria"
    ↪ value="perifericos">
  <label for="p">Periféricos</label>
  <input type="radio" id="c" name="categoria"
    ↪ value="componentes">
  <label for="c">Componentes</label>
</fieldset>

<select id="uf" name="uf">
  <option value="">Selecione...</option>
  <option value="PR">Paraná</option>
</select>
```

Rádios do mesmo grupo compartilham o name e diferem no value.

Caixa de seleção, área de texto e botões

```
<input type="checkbox" id="ok" name="aceite" value="sim">  
<label for="ok">Aceito receber novidades</label>
```

```
<textarea id="msg" name="mensagem" rows="5"></textarea>
```

```
<button type="submit">Cadastrar</button>  
<button type="reset">Limpar</button>  
<button type="button">Só com JavaScript</button>
```

Checkbox desmarcada não aparece na requisição. E o padrão de um `<button>` dentro de formulário é `submit`: esquecer o `type` recarrega a página sozinho.

Validação nativa

```
<input type="text" name="sku" required  
      pattern="[A-Z]{3}[0-9]{4}"  
      title="Três letras maiúsculas e quatro números">
```

```
<input type="number" name="quantidade" min="0" max="100"  
↪  step="1">
```

```
<input type="text" name="nome" minlength="3" maxlength="60">
```

O title é o texto que o navegador mostra ao recusar o valor do pattern.

Validação no cliente não é segurança

Toda essa checagem roda no navegador do usuário. Ele pode desativá-la pelas ferramentas de desenvolvedor, ou nem usar navegador: um `curl` monta a requisição à mão, com o conteúdo que quiser.

A validação nativa existe para dar retorno imediato e poupar viagem ao servidor.

O servidor **sempre** revalida tudo. Assunto da aula de PHP.

O que o HTML não faz

Aparência é CSS. Comportamento é JavaScript.

Por isso não se usa `<center>`, `<font>` nem `bgcolor`, todos obsoletos, e não se escolhe elemento pelo efeito visual que ele produz por padrão.

Teste de qualidade da marcação

Desative o CSS e leia o que sobrou. Se o documento continua fazendo sentido de cima a baixo, a estrutura está correta.

Sua primeira página

1. crie a pasta do exercício e abra-a no VS Code;
2. crie `index.html` e digite o esqueleto **à mão**;
3. acrescente um `h1` e um parágrafo;
4. salve e abra o arquivo no navegador. O endereço começa com `file://`;
5. editor e navegador lado a lado, F5 a cada alteração.

Crie também uma pasta `imagens/` ao lado do arquivo e referencie algo dentro dela com `src="imagens/arquivo.jpg"`. Imagem que não aparece é caminho errado.

No VS Code, ! seguido de Tab gera o esqueleto. Use depois de já saber escrevê-lo.

Inspecionar a árvore

F12, aba **Elements** ou **Inspetor**.

- passe o mouse pelos nós e veja o destaque na página;
- dê duplo clique e altere um texto: a página muda, o arquivo não;
- recarregue e a alteração some. Você editou a árvore em memória.

Experimento de recuperação de erro:

```
<p>Parágrafo <strong>com marcação <em>cruzada</strong>  
↪  aqui</em>.</p>
```

Compare o arquivo com a árvore exibida.

Formulário, e os dados na requisição

```
<form action="https://postman-echo.com/get" method="get">
  <label for="nome">Nome</label>
  <input type="text" id="nome" name="nome" required>
  <button type="submit">Enviar</button>
</form>
```

Três experimentos, um de cada vez:

1. remova o `name`: o valor some da URL;
2. troque para `post`: os dados saem da URL e vão para o corpo, visível na aba **Network**;
3. deixe o campo obrigatório vazio: nada chega à rede.

Validar no W3C

https://validator.w3.org/#validate_by_input

Erros comuns nesta altura:

- falta de `<!DOCTYPE html>` ou de `charset`;
- `<img>` sem `alt`;
- tag não fechada;
- id repetido no mesmo documento;
- `<li>` fora de `ul` ou `ol`.

Corrija até passar sem erros e sem avisos.

Entrega

Estes exercícios valem nota

Compõem o item *Exercícios em sala* da média. O **prazo de entrega** está na atividade correspondente na **UFPR Virtual**, e a entrega é por push no GitLab.

A tarefa é maior do que o tempo de aula: comece hoje, com o professor por perto, e termine durante a semana. A entrega é o próprio *fork* no GitLab, com os *commits* e o push feitos. **Não há link a enviar em lugar nenhum**: o professor recolhe os repositórios pelo nome do grupo.

Nome do grupo no padrão, alexkutzke como reporter e *fork* dentro do grupo. O que não é encontrado não é corrigido.

Onde está o enunciado

O enunciado completo fica no `README.md` do repositório da tarefa, e o endereço está na atividade correspondente na **UFPR Virtual**.

Individual ou em dupla. Em dupla, um faz o *fork* no próprio grupo e adiciona o colega como `developer`, com **Mob Programming** como forma de trabalho sugerida.

Atividade avaliativa

O uso de IA generativa para produzir o código **não é permitido**, conforme o plano de ensino. Consulte o material, a MDN e o professor.

O que a tarefa de hoje prepara

As páginas construídas hoje são a base da **Entrega 1 do trabalho prático**: catálogo de produtos em HTML5 e CSS3.

Semana que vem entra o CSS, e o que estiver bem estruturado hoje vai ser fácil de estilizar depois.

Resumo

- o servidor envia texto; o navegador monta a árvore que exibe;
- aninhamento errado não gera erro, gera árvore diferente da esperada;
- escolha o elemento pelo significado. `div` é o que sobra;
- a maior parte da acessibilidade vem de marcação correta;
- em formulário, `name` vai para a requisição, `id` liga o `label`, `method` decide entre URL e corpo;
- validação nativa serve ao usuário. O servidor revalida;
- sem CSS, o documento ainda precisa fazer sentido.